



OPS 베이커리 쇼핑몰 아키텍처 리포트

모놀리식 구조에서 모던 풀스택 시스템으로의 진화

개발 기간: 2025년 5월

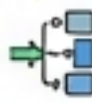
팀 구성: 홍상진 (Team Leader), 김인성, 김나연, 남제근

시스템 기반 및 기술 스택

프론트엔드

- **Core:** React (Vite) 
- **State:** Redux Toolkit 
- **Routing:** React Router v6 (SPA 방식 구현) 
- **Network:** Axios 

백엔드

- **Server:** Node.js, Express.js 
- **Routing:** /api Vite Proxy 설정 

데이터베이스 및 인증

- **DB:** MySQL 
- **인증:** JWT (AccessToken + RefreshToken) 
- **보안:** bcryptjs 비밀번호 암호화 

외부 연동

- **결제:** 카카오페이 API (준비/승인 프로세스) 

듀얼 엔진 개발 방법론



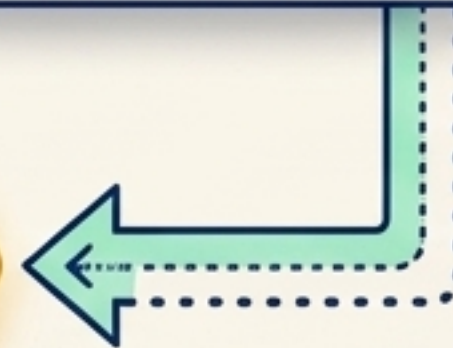
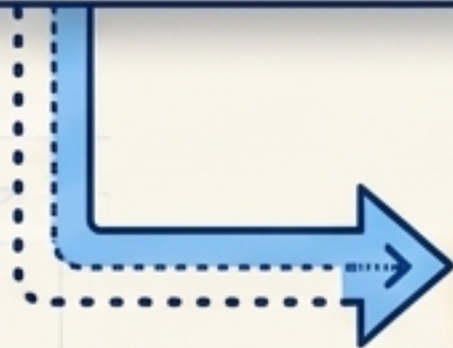
Claude AI 파트너십 (AI 기반 구조 최적화)

- ◆ • **동시 분석:** OPS 및 벤치마크 소스 코드 동시 업로드 및 구조 비교
- ◆ • **즉각적 이식:** 분석 결과를 OPS 환경에 맞춘 최적화 코드로 즉시 생성
- ◆ • **디버깅:** 에러 메시지 분석을 통한 신속한 원인 파악 및 수정



ShoppY 패턴 벤치마킹 (검증된 아키텍처 도입)

상태 관리	initForm 헬퍼 함수 (회원가입 폼 관리)
유효성 검사	useRef 기반 클라이언트 Validation
비즈니스 로직	cartItemsCheck (장바구니 중복 방지)
보안 아키텍처	JWT 이중 구조 및 Axios 인터셉터 기반 자동 재발급



**OPS Bakery
Core Architecture**

거대 단일 파일에서 모듈형 아키텍처로의 전환

App.jsx
(1,294줄)

기존 문제점: App.jsx 병목 현상

- Header, Footer, 라우팅, 페이지 뷰가 단일 파일에 집중
- <a href> 태그 사용으로 인한 불필요한 풀 리로드 (SPA 이점 상실)

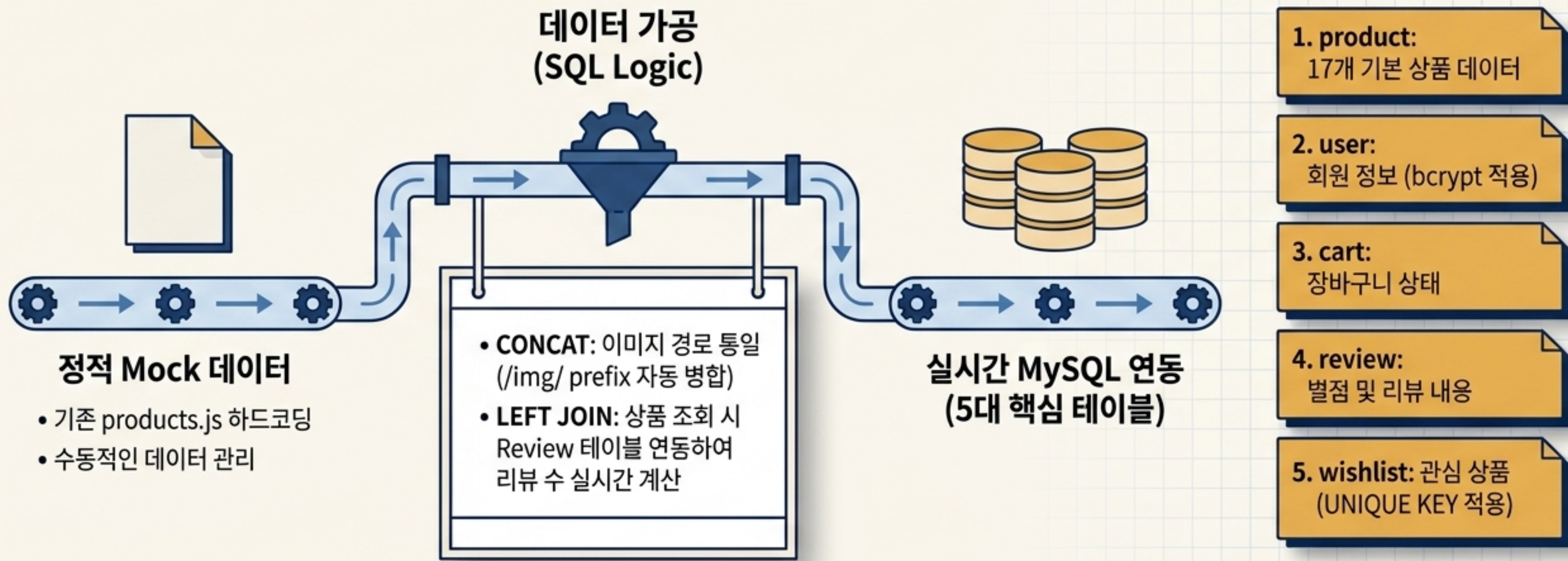


App.jsx (53줄) - 라우터 전용

- **Layout 컴포넌트:**
Header.jsx, Footer.jsx
- **Page 단위:** Home, ShopList, Detail, About, Member
 - **Micro 컴포넌트:**
ProductCard, Product

**라우팅 개선: <Link> 컴포넌트 전환으로
완벽한 SPA 렌더링 구현 ➡**

정적 데이터에서 동적 관계형 DB로의 마이그레이션

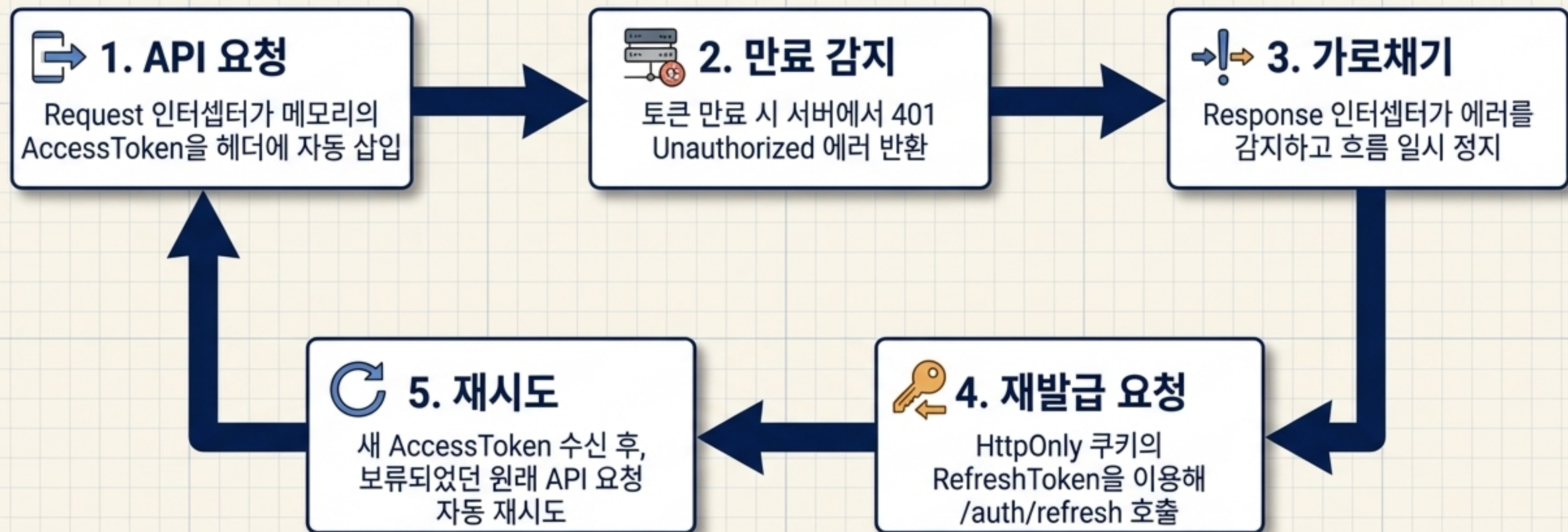


JWT 이중 토큰 기반 인증 아키텍처

단일 토큰 구조의 보안 취약점을 해결하기 위해 역할을 완벽히 분리한 이중 세션 관리 시스템 도입.

구분	AccessToken ⚡	RefreshToken 🗝️
핵심 역할	API 요청 시 권한 인증	AccessToken 만료 시 재발급
유효 기간	15분 (탈취 리스크 최소화)	7일 (사용자 편의성 유지)
저장 위치	Redux 메모리 (XSS 공격 방어)	HttpOnly 쿠키 (JS 접근 원천 차단)
검증 로직	verifyToken 미들웨어에서 매 요청 검사	POST /api/auth/refresh 에서만 검증

무중단 세션 복구 파이프라인 (Axios Interceptor)



새로고침 방어 로직 (App.jsx)

- restoreLogin: 페이지 새로고침 시 로그인 상태 자동 복구
- authChecked: 복구 완료 전까지 렌더링을 보류하여 UI 깜빡임 원천 차단

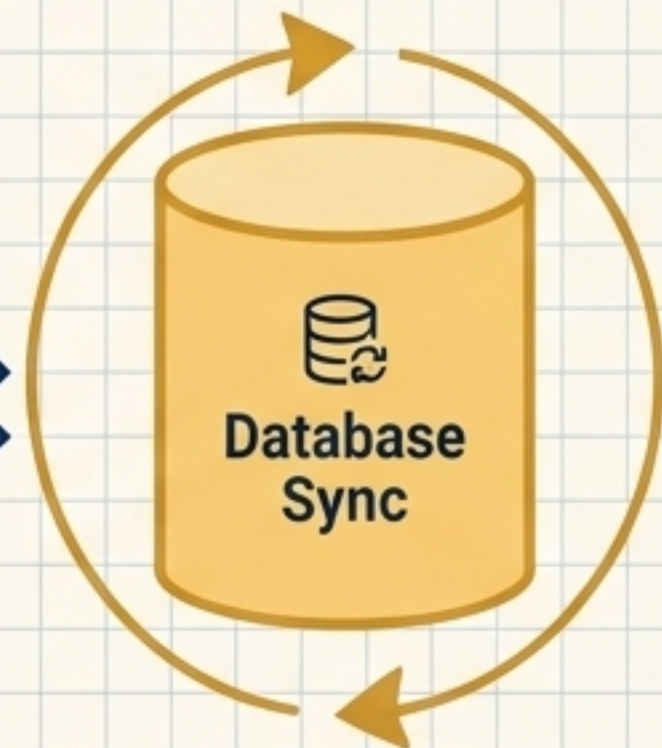
영구적 사용자 상태 관리 (장바구니 및 관심상품)

장바구니 로직 (모듈화 및 계산)

- 유틸리티 분리: `cartItemsCheck`, `updateCartItemsQty` 등 장바구니 비즈니스 로직을 `utils/cart.js`로 분리
- 스마트 결제: `getTotalPrice` 및 `getDeliveryFee` (3만원 이상 무료 배송 자동 계산)
- DB 연동: 추가 시 중복 감지 후 수량 자동 증가, 새로고침 시 헤더 배지 카운트 완벽 복원

관심상품 (데이터 무결성)

- 중복 방지 설계: DB 스키마에 UNIQUE KEY(uid, pid)를 적용하여 데이터 무결성 보장
- 토글 인터페이스: `POST /api/wishlist` 단일 엔드포인트로 찜 추가/취소 자동 전환
- 초기화 최적화: 페이지 진입 시 `GET /api/wishlist/check`로 하트 상태(♡/♥) 즉시 렌더링



결제 통합 퍼널 (카카오페이 API)



Order.jsx (결제 준비)

배송지 입력 및 약관 동의
→ 클라이언트 데이터
검증 완료



POST /api/kakao/ready

카카오페이 결제 준비
API 호출
→ 결제 고유 번호(tid) 및
리다이렉트 URL 확보



GET /api/kakao/approve

사용자 결제 진행
→ 결제 승인 API 호출 및
최종 결제 완료 처리



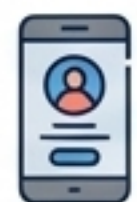
PayResult.jsx (트랜잭션 종료)

결과 페이지 렌더링
→ 성공 시 장바구니 DB
자동 비우기 처리

시스템 아키텍처 진화 요약 (Before & After)

아키텍처 요소	기존 구현 (Monolith & Static)	최종 구현 (Modern & Dynamic)
프론트엔드 구조	1,294줄의 단일 App.jsx 병목	책임이 분리된 53줄 라우터 및 모듈형 컴포넌트
라우팅 방식	<a> 태그 사용 (화면 풀 리로드)	React Router <Link> (완벽한 SPA)
데이터 관리	products.js 하드코딩 파일	MySQL DB 기반 실시간 데이터 연동
세션 스토리지	로컬 스토리지 (XSS 취약점 존재)	Redux 메모리 + HttpOnly 쿠키
세션 유지	새로고침 시 로그인 즉시 풀림	RefreshToken 기반 무종단 자동 복구
비밀번호 보안	데이터베이스에 평문 그대로 저장	bcrypt 단방향 암호화 적용
장바구니 상태	Redux 로컬 상태 (브라우저 종료 시 증발)	DB 영구 저장 및 헤더 배지 실시간 동기화

프로덕트 로드맵 및 향후 과제



Step 1: 사용자 경험 확장 (마이페이지 및 반응형 UI)

- 주문 내역 상세 조회 및 회원 정보 수정 기능 추가
- 모바일 및 태블릿 환경에 완벽히 대응하는 반응형(Responsive) 웹 디자인 적용



Step 2: 백엔드 보안 강화 (인프라 방어 로직)

- SQL Injection 방지 시스템 고도화
- Rate Limiting 도입을 통한 무차별 대입 공격(Brute Force) 및 DDoS 방어



Step 3: 프로덕션 배포 (CI/CD 및 서버 호스팅)

- AWS 또는 클라우드 플랫폼을 활용한 프론트엔드/백엔드 배포 환경 구축
- 무중단 서비스 운영을 위한 파이프라인 설계